

Experiment ‘p_13_8_recursive_largest_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_8_recursive_largest_array

Natural language statement: null

Method signature: p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)

Ensures

- `largest == find_largest(inputs[..])`

Requires

- `inputs.Length >= 1`

Functional Code Given

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)`

The method should respect the following contract:

`requires inputs.Length >= 1, ensures largest == find_largest(inputs[..])`

The contract uses the following dafny code:

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
  }
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnP3txdTl9DOLA9rECphojclkl17y

Model: gpt-5.5-2026-04-23

Created at: 1780666781

Finish reason: stop

Usage: 524 tokens in, and 6642 tokens out

```
{
  var i := inputs.Length - 1;
  largest := inputs[i];

  var init := inputs[i..];
  assert |init| == 1;
```

```

assert init[0] == inputs[i];
assert find_largest(init) == inputs[i];
assert largest == find_largest(inputs[i..]);

while i > 0
  invariant 0 <= i < inputs.Length
  invariant largest == find_largest(inputs[i..])
  decreases i
{
  var subLargest := largest;
  i := i - 1;

  assert 0 <= i < inputs.Length;
  assert i + 1 < inputs.Length;
  assert subLargest == find_largest(inputs[i + 1..]);

  var s := inputs[i..];
  assert |s| > 1;
  assert s == [s[0]] + s[1..];
  assert s[0] == inputs[i];
  assert s[1..] == inputs[i + 1..];
  assert find_largest(s) == (if s[0] >= find_largest(s[1..]) then s[0] else find_largest
    ↪ (s[1..]));
  assert find_largest(inputs[i..]) == (if inputs[i] >= subLargest then inputs[i] else
    ↪ subLargest);

  if inputs[i] >= subLargest {
    largest := inputs[i];
  } else {
    largest := subLargest;
  }

  assert largest == (if inputs[i] >= subLargest then inputs[i] else subLargest);
  assert largest == find_largest(inputs[i..]);
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert largest == find_largest(inputs[..]);
}

```

Final Program

```

function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];

```

```

    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}

method p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)
    requires inputs.Length >= 1
    ensures largest == find_largest(inputs[..])
{
    var i := inputs.Length - 1;
    largest := inputs[i];

    var init := inputs[i..];
    assert |init| == 1;
    assert init[0] == inputs[i];
    assert find_largest(init) == inputs[i];
    assert largest == find_largest(inputs[i..]);

    while i > 0
        invariant 0 <= i < inputs.Length
        invariant largest == find_largest(inputs[i..])
        decreases i
    {
        var subLargest := largest;
        i := i - 1;

        assert 0 <= i < inputs.Length;
        assert i + 1 < inputs.Length;
        assert subLargest == find_largest(inputs[i + 1..]);

        var s := inputs[i..];
        assert |s| > 1;
        assert s == [s[0]] + s[1..];
        assert s[0] == inputs[i];
        assert s[1..] == inputs[i + 1..];
        assert find_largest(s) == (if s[0] >= find_largest(s[1..]) then s[0] else find_largest
            ↪ (s[1..]));
        assert find_largest(inputs[i..]) == (if inputs[i] >= subLargest then inputs[i] else
            ↪ subLargest);

        if inputs[i] >= subLargest {
            largest := inputs[i];
        } else {
            largest := subLargest;
        }

        assert largest == (if inputs[i] >= subLargest then inputs[i] else subLargest);
        assert largest == find_largest(inputs[i..]);
    }

    assert i == 0;
    assert inputs[i..] == inputs[..];

```

```
    assert largest == find_largest(inputs[...]);  
}
```

Total Token Usage

Input tokens: 524

Output tokens: 6642

Reasoning tokens: 6141

Sum of ‘total tokens’: 7166

Experiment Timings

Overall Experiment started at 1780666780499, ended at 1780666944671, lasting 164172ms (164.17 seconds)

Iteration #1 started at 1780666780500, ended at 1780666944521, lasting 164021ms (164.02 seconds)